

Repository Skills

Reviewable guidance in `.github/skills/`

Delivery rule

```
.github/skills/  
└─ api-design/  
   └─ SKILL.md
```

Keep skills in the repository so the team can review, version, own, and test them with the code they affect.

Preflight and fallback

Verify current documentation and customer policy for the repository, surface, data, tools, meter, and reviewer. Use a non-sensitive sandbox.

If access is unavailable, create the same artifact and use it as a manual checklist. Do not move the skill or bypass policy.

A skill is an executable team agreement

A skill gives an agent a repeatable procedure. The repository gives the team a way to review, version, and improve that procedure.

Use a skill when the work has stable steps and a recognizable outcome. Do not make a skill for a one-off prompt, a vague aspiration, or a policy that belongs in repository instructions.

Guidance type	Best home
Always true for the repository	Instructions
Repeatable task procedure	Skill
One specialist point of view	Custom agent

Write the trigger as carefully as the procedure

The trigger decides when the procedure appears. A broad trigger creates noise; a narrow trigger lets the model miss relevant work.

When to use

Use when creating or changing REST routes under `src/api/`.

Do not use for internal helpers or generated clients.

Name the work, its boundary, and any exclusion that protects the workflow. Then make each procedure step observable enough for a reviewer to check.

A skill should fail visibly

Skills need preconditions and stop conditions, not just happy-path instructions.

Situation	Expected behavior
Required tests are absent	Report the gap and ask for a decision
Input is outside the named area	Stop and redirect to the right procedure
Command fails	Surface the output and do not claim success
Access is missing	Use the documented manual fallback

That is what turns a prompt into a process someone can trust.

A useful skill

When to use

Creating or changing REST routes under `src/api/`.

Procedure

1. Read the route and its tests.
2. Validate input at the boundary.
3. Add focused success and failure tests.
4. Request review before merging.

Add preconditions, validation, fallback, and an owner.

Review a community candidate

Check source, owner, license, version, instructions, scripts, dependencies, network use, data flow, and repository fit. Popularity is not approval.

Never run an unfamiliar bundled script merely to inspect it.

Compose minimally

Need	Use
Repository convention	Instructions
Repeatable procedure	Skill
Specialized role	Custom agent
Approved external capability	MCP
Shared interactive artifact	Canvas
Distributed customizations	Plugin

Lab

Create two skills, review a candidate, and apply one procedure manually in the sandbox.